

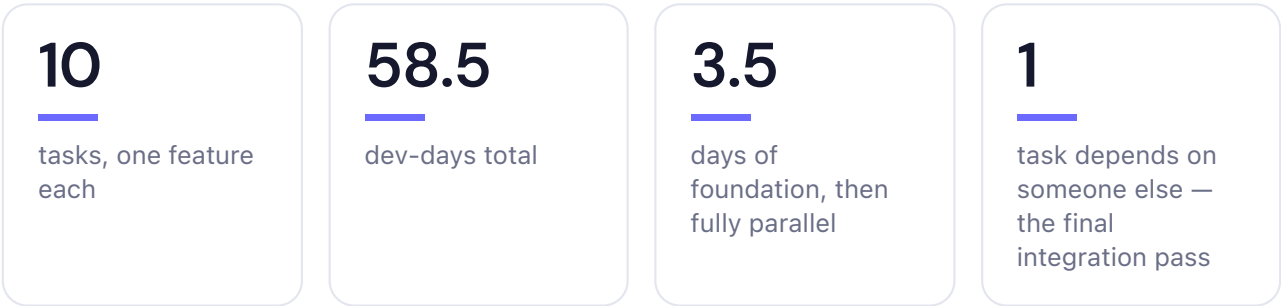
DELIVERY PLAN

SPRINTS — WORK BREAKDOWN

Internal document · team of 4

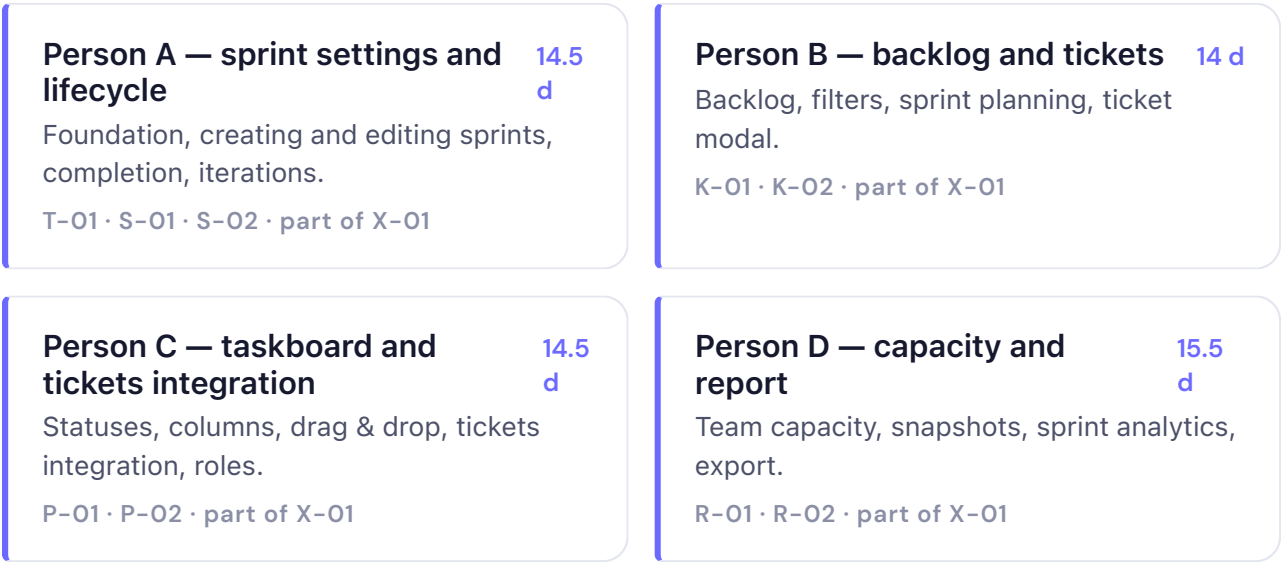
Sprints feature: 10 tasks

The split is by feature, not by layer: one person owns one feature from database to screen, including its tests. After a shared foundation, all four tracks run in parallel and barely touch each other.



Ownership

Everyone is full-stack: the feature owner writes the database, the API, the screen and the tests for it. QA and design are roles inside the team — nothing is handed over at the end. The API contract and mock responses are produced in T-01, so nobody waits on someone else's backend.



Tasks

Estimates are ideal dev-days for one person, including tests and review. Titles and descriptions are ready to paste into a ticket — **Done when** is the acceptance criteria list.

Every task carries a **Mockup**: reference to a screen or state in the interactive mockup **Sprints — labelled nav** (attached HTML). The mockup is the source of truth for layout, columns, tones and behaviour — not the description in this document.

T-01 Foundation: data model, API contract and screen shell

Person A

3.5 d

Sets up everything the other nine tasks build on. The sprint data model and its companions (Sprint, SprintTicket, Iteration, CapacityEntry) with migrations and seed data, a typed API contract covering all four tracks with mock responses alongside it, and the frontend shell: Boards group in the sidebar, sprint tabs, per-sprint deep links and the shared empty, loading and error states. The goal is that after 3.5 days nobody waits on anyone else's backend — everyone works against the contract and the mocks.

DONE WHEN

- Migrations are reversible and run on a clean database with seed data.
- The API contract covers every endpoint the other tasks need, with mock responses the frontend can switch on and off.
- /sprints opens with tabs and per-sprint deep links, including empty, loading and error states.
- Ownership of SprintTicket fields is written down: statuses are changed by P-01, sprint assignment by K-01.

Mockup: Sidebar → Boards, tabs Backlog / Taskboard / Capacity / Report Depends on: —

Layer: BE + FE

S-01 Sprint settings: create, edit and activate a sprint

Person A

5 d

The full sprint lifecycle up to completion. Backend: sprint CRUD with validation of name, dates and capacity, overlap checks against other sprints, and the rule that at most one sprint per project is active. Frontend: the new sprint dialog with inline validation and a submitting state, editing an active sprint with locked fields that must no longer change, and a sprint header showing dates and days remaining.

DONE WHEN

- A new sprint is created from the dialog; overlapping dates and missing required fields produce inline errors, not a toast.
- Activation is idempotent and never allows a second active sprint in the same project.
- Editing an active sprint changes dates and capacity; locked fields are visibly disabled with an explanation.
- Tests cover date validation, overlap and double activation.

Mockup: New sprint dialog + sprint header Depends on: T-01 Layer: BE + FE**S-02 Sprint completion and iteration history**

Person A

5 d

Closing a sprint without losing work: unfinished tickets are carried into the next sprint or returned to the backlog, the sprint is locked against edits, and the event is written to the audit log. The UI side is a confirmation dialog summarising what will be carried over, plus a toast with the number of moved tickets. On top of that the iterations screen: history of every sprint with its key numbers, reopening a closed sprint, and comparing two sprints.

DONE WHEN

- Completion is transactional: either everything is carried over and the sprint locks, or nothing happens.
- Before confirming, the dialog shows the exact number of unfinished tickets and the destination choice (next sprint or backlog).
- A closed sprint cannot be edited; reopening is audited with user and timestamp.
- The Iterations list shows every closed sprint with duration, completed and carried over.

Mockup: Complete sprint dialog + toast, Iterations Depends on: S-01 Layer: BE + FE

K-01 Backlog: browsing, filtering and sprint planning

Person B 7 d

The main planning screen. Backend: a backlog endpoint with sections, sorting, filters and pagination, story point sums per section, and transactional bulk assign/unassign to a sprint. Frontend: the backlog table with its columns, badge tones, priority and story points, multi-row selection, per-section collapse and Collapse all, sums in the section header, and drag & drop of tickets into the sprint with optimistic updates and rollback on failure.

DONE WHEN

- Filters, sorting and pagination work together and survive a reload (state in the URL).
- Multi-row selection plus one action adds or removes tickets from the sprint in a single transaction.
- Drag & drop moves a ticket between sections and into the sprint; a failure returns the card to its original position with a message.
- Story point sums per section and for the sprint update without reloading the whole list.
- A list of ~500 rows stays usable (virtualisation or pagination, measured).

Mockup: Sprints → Backlog (table, sections, Collapse all, drag into sprint) Depends on: T-01
Layer: BE + FE

K-02 Creating and editing a ticket from the sprint and the backlog

Person B 5.5 d

The new ticket modal, reachable from both places — the backlog and the taskboard — prefilling sprint and section from the context it was opened in. Fields: title, description, type, priority, story points, assignee, labels. Inline validation, a submitting state, and editing an existing ticket through the same modal. The backend part is a create/update endpoint with permissions and a change record.

DONE WHEN

- Opened from a sprint the modal prefills sprint and section; opened from the backlog it leaves the sprint empty.
- Field errors are inline; the modal does not close on a failed submit and does not lose entered data.
- Editing reuses the same modal and shows current values without flicker.
- A new ticket appears in the list without a full reload, in the correct position for the active sort.

Mockup: New task modal (from Backlog and from Taskboard) Depends on: K-01 Layer: BE + FE

Person C 8 d

DONE WHEN

- Mockup:** Sprints → Taskboard (Previous sprint as first column, cards, drag & drop, filters)

Person C 5 d

DONE WHEN

- Mockup:** Tickets → Sprint column and filter; Role switch → intern **Depends on:** T-01 **Layer:** BE + FE

Person D 4.5 d

DONE WHEN

- Page 4 of 6

- Team totals match the sum of taskboard cards for the same sprint.
- Empty state and the no-capacity-defined state are both implemented.
- The screen is hidden for roles without rights (wired to P-02).

Mockup: Capacity (workload per member) **Depends on:** T-01 **Layer:** BE + FE

R-02 Sprint report and analytics with export

Person D 9.5 d

Measuring the sprint. A daily snapshot of sprint state is written from day one, because a burndown without history does not exist. Aggregations build on it: velocity, carry-over, health, and distribution by member and type. The frontend has two tabs — Summary with the sprint recap, key numbers and the carried-over list, and Stats with burndown, velocity trend and distribution. Plus CSV and PDF export carrying the same data as the screen, with downloading and error states.

DONE WHEN

- The snapshot is written automatically every day and for the sprint completion day.
- Report numbers match the taskboard and capacity after the sprint is closed.
- Charts have an empty state for a sprint without history and do not block the rest of the screen from rendering.
- Export returns the same data set as the screen; a download failure is surfaced to the user.

Mockup: Report → Summary, Report → Stats, Report → Export menu **Depends on:** T-01

Layer: BE + FE

X-01 Integration pass: end-to-end flow, accessibility and polish

all (split) 5.5 d

The final task, where the four tracks come together as one flow. An integration E2E walks a single sprint from creation, through planning and the board, to capacity and the report. Alongside it: a keyboard alternative to drag & drop and a sane focus order, verification on Chrome, Safari and Firefox with a screen reader, a visual pass across every screen with microcopy and term tooltips (velocity, carry-over), and performance measurement at realistic volume. Split into four parts, one per person.

DONE WHEN

- One E2E scenario passes the whole flow across all four tracks in CI.
- Every drag & drop action has a keyboard alternative and announces the change to a screen reader.
- A per-browser audit report is recorded, with findings either closed or explicitly accepted.
- Alignment, tones and copy match the mockup; terms are explained in the UI.

Mockup: all screens (plan → board → capacity → report) **Depends on:** S-02, K-02, P-02, R-02

Layer: QA + design + FE

Load and sequence

PERSON	TASK SEQUENCE	DEMO WHEN FINISHED	DAYS
Person A	T-01 → S-01 → S-02 → X-01	A sprint from creation to completion, history in iterations	14.5
Person B	K-01 → K-02 → X-01	Filtered backlog, drag into sprint, new ticket from context	14
Person C	P-01 → P-02 → X-01	Ticket through the columns without conflict, sprint visible on tickets	14.5
Person D	R-01 → R-02 → X-01	Overloaded member flagged, report with velocity and burndown	15.5

Definition of done

- The feature owner writes its tests too — nothing is handed to QA at the end.
- Code reviewed by one other person on the team.
- Empty, loading and error states implemented — not just the happy path.
- Keyboard and focus work; drag & drop has an alternative.
- Screen checked side by side against the mockup before merging.
- No temporary data and no feature flags left switched off.

Risks

- The foundation is the only bottleneck**

T-01 blocks all four people. It goes first, paired if needed, and its scope does not grow mid-flight.
- The backlog / taskboard boundary**

Both touch SprintTicket. Status belongs to P-01, sprint assignment to K-01 — agreed in T-01, no overlap.
- Burndown depends on history**

R-02 must write the daily snapshot from day one, otherwise the first sprint has no chart.
- Drag & drop is the most expensive part**

It sits in K-01 and P-01 with two different people. If it overruns, ship status changes from a menu first.

ASSUMPTIONS

The team is four people in total — QA and design are roles inside the team, not extra headcount. Tasks are deliberately coarse: with AI support the owner breaks them into subtasks in the ticket. Azure DevOps sync is out of scope. The existing tickets model is extended, not replaced. Estimates exclude devops, staging environments and historical data migration.